

API's

General knowledge

Path

"Path" here refers to the last part of the URL starting from the first `/`.

So, in a URL like:

```
https://example.com/items/foo
```

...the path would be:

```
/items/foo
```

Info

A "path" is also commonly called an "endpoint" or a "route".

While building an API, the "path" is the main way to separate "concerns" and "resources".

Operation

"Operation" here refers to one of the HTTP "methods".

One of:

- `POST`
- `GET`
- `PUT`
- `DELETE`

...and the more exotic ones:

- `OPTIONS`
- `HEAD`
- `PATCH`
- `TRACE`

In the HTTP protocol, you can communicate to each path using one (or more) of these "methods".

When building APIs, you normally use these specific HTTP methods to perform a specific action.

Normally you use:

- `POST` : to create data.
- `GET` : to read data.
- `PUT` : to update data.
- `DELETE` : to delete data.

So, in OpenAPI, each of the HTTP methods is called an "operation".

We are going to call them "operations" too.



@decorator Info

That `@something` syntax in Python is called a "decorator".

You put it on top of a function. Like a pretty decorative hat (I guess that's where the term came from).

A "decorator" takes the function below and does something with it.

In our case, this decorator tells **FastAPI** that the function below corresponds to the path `/` with an operation `get`.

It is the "path operation decorator".

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

This is a Python function.

It will be called by **FastAPI** whenever it receives a request to the URL `/` using a `GET` operation.

In this case, it is an `async` function.

You could also define it as a normal function instead of `async def`:

Flask

- Lightweight and flexible web framework for Python.
 - Popular since 2010 for its simplicity and ease of use.
 - Minimal built-in API features; relies on extensions:
 - `Flask-RESTful` for building REST APIs.
 - `Flask-SQLAlchemy` for database integration.
 - Does **not** support `async / await`.
 - Can struggle with handling many concurrent connections efficiently.
 - Good choice for simple or small-scale APIs.
-

Django REST Framework (DRF)

- Toolkit for building APIs with Django.
 - Follows the "batteries-included" philosophy — comes with many built-in features:
 - Serialization
 - Query parameter parsing
 - Authentication and permissions
 - Powerful and feature-rich, but can feel **heavy** for small/simple APIs.
 - Does **not** support `async / await`.
 - Best suited for projects that need robust built-in components and tight Django integration.
-

FastAPI

- Modern framework focused on building APIs efficiently and intuitively.
- Supports modern Python features:
 - `async / await` for asynchronous programming.
 - Type hints for validation, docs, and editor support.
- Built on:
 - `Starlette` for web layer.
 - `Pydantic` for data validation and serialization.
- Advantages:
 - **High performance** — comparable to Node.js and Go.
 - **Automatic interactive API documentation** (Swagger UI and ReDoc).
 - **Simple and flexible routing** system.
 - **Dependency injection** support.

- **Automatic data validation** using type hints.
 - Best choice for fast, scalable APIs with modern Python practices.
-

Fast API

With `async / await`, the server can say:

“Okay, this task is waiting... in the meantime, let me handle someone else's request.”

So:

- `async def` = defines a function that can pause and let other code run.
- `await something()` = you're waiting for something (e.g., a request or a slow operation), and Python uses that time to multitask.

✅ FastAPI uses this for better **performance and scalability**.

🔧 FastAPI uses Pydantic to **automatically validate and parse incoming API requests**.

Pydantic uses type hints to know what valid data looks like.

Example:

```
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int
```

Now you can say:

```
User(name="Alice", age=25) ✅
User(name="Bob", age="not-a-number") ❌
```

Technical Details

`FastAPI` is a class that inherits directly from `Starlette`.

You can use all the `Starlette` [\[↔\]](#) functionality with `FastAPI` too.

Example of GET endpoint definition:

```
@app.get("/items/{item_id}")

def read_item(item_id: int, q: Union[str, None] = None):

    return {"item_id": item_id, "q": q}
```

What this code does:

- `@app.get("/items/{item_id}")`:

This is a **route decorator** that defines a **GET endpoint**. It means:

- When someone sends a `GET` request to `/items/42`, this function is called with `item_id=42`.
- `item_id` is part of the **URL path**, so it's a **path parameter**.
- `def read_item(item_id: int, q: Union[str, None] = None)`:
 - The function that handles the request.
 - `item_id` must be an **integer**. FastAPI will **automatically validate** this.
 - `q` is an optional **query parameter**, meaning:
 - `/items/42?q=hello` would set `q="hello"`.
 - `/items/42` would set `q=None`.
 - The `Union[str, None]` (or `Optional[str]`) means it can be either a `str` or `None`.
- `return {"item_id": item_id, "q": q}`:

This is the **JSON response** sent back to the client.

FastAPI docs interface

5. What You See in the Docs Interface

When you go to your **FastAPI docs page** (<http://localhost:8000/docs>), the interface (Swagger UI) shows:

Try it out button

You can:

1. Click `GET /items/{item_id}`

2. Enter a value for `item_id` (e.g. 123)
3. Optionally enter a query value `q` (e.g. hello)
4. Click **Execute**

FastAPI generates a "schema" with all your API using the **OpenAPI** standard for defining APIs.

"Schema" ¶

A "schema" is a definition or description of something. Not the code that implements it, but just an abstract description.

API "schema" ¶

In this case, [OpenAPI](#) is a specification that dictates how to define a schema of your API.

This schema definition includes your API paths, the possible parameters they take, etc.

Data "schema" ¶

The term "schema" might also refer to the shape of some data, like a JSON content.

In that case, it would mean the JSON attributes, and data types they have, etc.